

AI Companion Video Call & Streaming — Technical Documentation

1. Overview

The **AI Companion Video Call & Streaming System** is an end-to-end platform that enables users to interact with intelligent AI companions through **real-time video, chat, and voice**.

The system leverages **FastAPI, Next.js, Gemini API, WebRTC**, and **WebSockets** to achieve low-latency, human-like communication between a user and an AI avatar.

All components are **free-tier compatible** and deployable on both **local and cloud environments** without modification.

2. Objectives

- Create a **real-time two-way video calling** interface using **WebRTC**.
- Enable **AI-powered conversations** using **Gemini API** for intelligent text responses.
- Integrate **Speech-to-Text (STT)** and **Text-to-Speech (TTS)** for natural voice interactions.
- Maintain **persistent AI memory** using **MongoDB**, allowing context continuity between sessions.

- Implement **recording** of live sessions for review or testing.
 - Use **Redis** as the high-speed in-memory signaling layer for real-time state synchronization.
 - Ensure full **free-tier and offline functionality** for hackathon and future production use.
-

3. System Architecture Summary –

Frontend (Next.js + TypeScript):

Handles the UI, video rendering, chat display, and WebRTC peer connection logic.

Backend (FastAPI):

Acts as the core controller — manages signaling, room creation, AI responses, recordings, and database interactions.

Databases & Services:

- **Redis** — Temporary storage and pub/sub messaging for active calls.
 - **MongoDB** — Persistent chat and memory storage.
 - **Gemini API** — Processes natural language and returns AI-generated replies.
 - **WebRTC** — Enables direct media streaming between peers.
 - **STT/TTS Layer** — Converts voice ↔ text using local browser APIs or external services.
-

4. Component Explanations

4.1 Redis (Real-Time Signaling & Session Management)

How it works:

- Redis stores temporary room and peer connection states (roomId, userId, ICE candidates).
- When a user joins, the backend emits a WebSocket “join” event. Redis publishes this to all connected clients in that room.
- Redis pub/sub ensures instant propagation of **offer**, **answer**, and **candidate** messages for WebRTC negotiation.
- Once a session ends, Redis automatically clears the cached state to free memory.

Why Redis:

- Sub-millisecond latency.
 - Ideal for transient data (signaling doesn’t require permanent storage).
-

4.2 MongoDB (Persistent Memory & Chat Storage)

How it works:

- Every chat message (roomId, user, message, timestamp) is stored in MongoDB.
- When a user rejoins with the same roomId, MongoDB fetches previous context so the AI can continue the conversation seamlessly.

- The backend includes a lightweight memory retriever that passes prior context to Gemini before generating the next response.

Why MongoDB:

- JSON-like document structure aligns perfectly with chat data.
 - Runs both locally (mongodb://localhost:27017) and in the cloud (MongoDB Atlas free-tier).
 - Zero configuration switch between environments.
-

4.3 WebRTC (Media & Connection Layer)

How it works:

- The frontend captures **user camera and microphone** via the browser's `getUserMedia()` API.
- It sends an **offer SDP** to the backend via WebSocket.
- The backend broadcasts this to the companion peer (AI or demo mirror).
- The companion responds with an **answer SDP**, establishing a peer-to-peer connection.
- Both sides exchange **ICE candidates** using Google's free **STUN server**:
- `stun:stun.l.google.com:19302`
- Once the negotiation succeeds, the video and audio stream directly between browsers without server relay (zero latency).

Why WebRTC:

- True peer-to-peer communication.
 - Secure (DTLS + SRTP).
 - Native browser support — no plugin needed.
-

4.4 Gemini API (Conversational Brain)

How it works:

- The backend's `/api/chat` route handles incoming text messages.
- Each message is sent to **Google Gemini API** via authenticated HTTP request.
- Gemini processes the prompt (including previous MongoDB chat context) and returns a structured JSON response.
- The frontend ChatPanel displays the AI reply in real-time and triggers TTS for audio output.

Why Gemini:

- Lightweight, fast inference.
 - Excellent contextual understanding.
 - Simple JSON response structure ideal for chat rendering.
-

4.5 STT (Speech-to-Text)

Local (Browser):

- Uses the **Web Speech API** (`window.SpeechRecognition`) to capture and transcribe user speech.
- Converts spoken input to text, which is then sent to Gemini for processing.

Cloud (Optional):

- If deployed online, can use **Google Cloud Speech-to-Text** (free-tier 60 minutes/month) for improved accuracy.

Integration Flow:

1. User presses mic button → `SpeechRecognition` starts.
 2. Transcribed text sent to backend → Gemini API.
 3. Response text displayed → passed to TTS for playback.
-

4.6 TTS (Text-to-Speech)

Local (Browser):

- Uses **Web Speech Synthesis API** (`speechSynthesis.speak()`) to generate natural voice playback.

Cloud (Optional):

- Can integrate **ElevenLabs API** for realistic avatar voices (optional and free for limited use).

Flow:

1. Gemini returns text reply.
 2. TTS converts reply → voice stream.
 3. Audio plays via browser output in sync with the AI avatar animation.
-

4.7 Recording Module

How it works:

- The frontend uses **MediaRecorder API** to capture both video and audio streams.
- When the user stops recording, a Blob is generated.
- This Blob is uploaded to `/api/video/recordings`, which stores it locally or in a cloud bucket (future-ready).

Why it matters:

- Allows reviewing calls and debugging AI–user interactions.
-

5. API Design

Method	Endpoint	Purpose
POST	<code>/api/video/rooms</code>	Create a new room and assign IDs.
GET	<code>/api/video/rooms/{roomId}</code>	Validate or fetch room details.

GET	/api/webrtc/config	Provide STUN configuration for peer connection.
GET	/api/companions	Proxy companion data from Persona Fetcher API.
POST	/api/video/recordings	Accept and save recorded session.
WS	/signal	Manage signaling events (join, offer, answer, candidate, leave, end).

6. Data Flow

- 1. User Action:** Opens the companion page → selects avatar → clicks “Start Call.”
 - 2. Backend:** Creates a unique room and returns configuration.
 - 3. WebRTC:** Begins peer connection → exchanges ICE candidates via Redis-backed signaling.
 - 4. User Speech:** STT converts speech → text → sent to Gemini API.
 - 5. AI Response:** Gemini returns text → TTS converts to voice → plays to user.
 - 6. MongoDB:** Logs conversation context for memory retention.
 - 7. End Call:** Recording uploaded (optional) → Redis clears session data.
-

7. Frontend–Backend Integration

- WebSocket and REST APIs combined for minimal latency.

- Frontend handles all UI rendering and browser-native media capture.
 - Backend ensures security, state management, and response generation.
 - Every call is **stateless beyond active sessions**, ensuring scalability.
-

8. Security & Error Handling

- WebRTC uses **DTLS + SRTP** encryption by default.
 - FastAPI routes validate all inputs with **Pydantic models**.
 - Recording uploads are limited by server-side file size checks.
 - Redis keys auto-expire to prevent data leaks.
 - MongoDB collections index by roomId for fast retrieval and access control.
-

9. Local Setup Instructions

Backend:

```
cd backend
```

```
python -m venv venv
```

```
venv\Scripts\activate
```

```
pip install -r requirements.txt
```

```
uvicorn main:app --reload
```

Frontend:

cd frontend

npm install

npm run dev

Access:

- **Frontend:** <http://localhost:3000>
 - **Backend:** <http://localhost:8000>
-

10. Conclusion

This version of **AI Companion Video Call & Streaming** represents a fully engineered, deployable solution built for both local testing and cloud scalability.

The platform extends far beyond foundational requirements by integrating persistent **AI memory**, advanced **text-to-speech (TTS)** and **speech-to-text (STT)** modules, and **real-time session recording**—while maintaining free-tier compatibility and enforcing a robust, modular design approach.

The system demonstrates a complete, production-ready pipeline from **voice input to AI-driven conversational output**, embodying the true objective of a human-like AI companion.
